Beyond Binary: Unified Detection and Attribution of AI-Generated Code via Hierarchical Family-Aware Learning

Research Proposal — NeurIPS 2026

Dao Sy Duy Minh, Huynh Trung Kiet, Tran Chi Nguyen
and collaborators in **HCMUS_TheFangs**

Ho Chi Minh City University of Science (HCMUS)
VNUHCM

April 1, 2026

Outline

1. Problem: Why Binary Detection is Not Enough
2. Benchmarks: Three Complementary Challenges
3. Method 1: SpectralCode (Completed)
4. Results: SpectralCode (Completed)
5. Method 2: HierTreeCode (In Progress)
6. Evaluation Plan & Timeline
7. Experiment Insights

Existing Benchmarks: Binary-Only and Limited Scale

Classic Benchmarks (Binary Detection Only)

Benchmark	Size	Lang	Models	Attrib?	Adv?
GPTSniffer '24	7.4K	1	1	✗	✗
Whodunit '24	1.6K	1	1	✗	✗
CodeGPTSensor '25	1.1M	2	1	✗	✗
CodeMirage '25	210K	10	10	✗	✗
CoDET-M4 ACL'25	500K	3	5	✓	✗
DROID EMNLP'25	1.06M	7	43	✗	✓
AICD EACL'26	2.05M	9	77	✓	✓

Older benchmarks: 1 model, 1 language, **binary only**. Newer: multi-model, multi-task, multi-language.

Why Classic Benchmarks Are Obsolete

- ✗ **Single model**: GPTSniffer/Whodunit only test 1 LLM
- ✗ **Binary only**: Cannot attribute to specific model
- ✗ **No OOD**: Train/test on same language
- ✗ **No adversarial**: No DPO-tuned evasion
- ✗ **Saturated**: Fine-tuned models >98% F1

Binary is Solved (CoDET-M4)

UniXcoder: 98.65% F1
SpectralCode (ours): 99.06% F1
⇒ No room for improvement

The Real Challenges Remain

Beyond Binary: Three Open Problems

Authorship Attribution: *Which* LLM generated it? (6–12 classes)
Best SOTA (UniXcoder on CoDET-M4): only **66.33%** F1 — a 32-point drop from binary.
Classic benchmarks *cannot even evaluate this* — they have no multi-class labels.

OOD Generalization: Train on Python/Java/C++ → test on Go/Rust/PHP?
AICD T1: validation 99.5% → test **29.8%** F1. Catastrophic collapse across *all* methods.
Classic benchmarks test on the same language they train on — OOD is invisible.

Adversarial Robustness: DPO-tuned models that mimic human style?
DROID includes 157K adversarial samples. DroidDetect recall drops from 98% → 92% on adversarial code.
No classic benchmark has adversarial or machine-refined data.

Our vision: A unified method evaluated across **3 modern benchmarks**, **3.6M samples**, covering binary + attribution + OOD + adversarial — the problems that *classic benchmarks cannot even measure*

The Bigger Picture: From Detection to Understanding

The Evolution of the Threat

2023 "Is this code written by AI?" → Binary detection (**solved**: >99% F1)

2024 "Which model family generated it?" → Attribution (**hard**: best 66% F1, 6-class)

2025 "Can the model fool the detector?" → DPO-tuned adversarial code (DROID, EMNLP'25)

2026 "Does it generalize to unseen languages?" → OOD collapse: 99% → 30% (AICD, EACL'26)

Why Current Methods Fail at Attribution

- LLMs form **family trees**: Nxcode = fine-tuned Qwen1.5, CodeLlama = fine-tuned Llama
- Fine-tuned models inherit parent's "stylometric DNA" — nearly identical coding patterns
- Standard classifiers treat all classes as **equidistant** → 40%+ confusion between parent-child pairs
- Qwen1.5 F1 = 0.41 (near random for 6-class)

Our Key Insight

The family tree of LLMs is not a nuisance — it is **structured prior knowledge**.

Models within the same family should be *closer but separable* in embedding space.

⇒ **Encode genealogy as distance constraints**

Three Benchmarks — Why These Three?

	CoDET-M4 Orel et al., ACL Findings 2025	**DroidCollection** Orel et al., EMNLP 2025	**AICDBench** Orel et al., EACL 2026
Scale	500K samples	1.06M samples	2.05M samples
Languages	3 (Py, Java, C++)	7 languages	9 languages
Models	5 generators	43 models, 11 families	77 models, 11 families
Sources	CF, GitHub, LeetCode	General, Algo, Research	Algo, Research, General
Binary	✓ (saturated 99%)	✓ (saturated 97%)	✓ + **4-split OOD**
Attribution	✓ 6-class	—	✓ 12-class family
Refined	✗	✓ **3 scenarios**	✓ (hybrid class)
Adversarial	✗	✓ **DPO 157K pairs**	✓ (in Task 3)

★ CoDET-M4: Attribution

Best for fine-grained authorship (6-class). Known Nxcode↔Qwen confusion. Per-lang/source breakdowns.

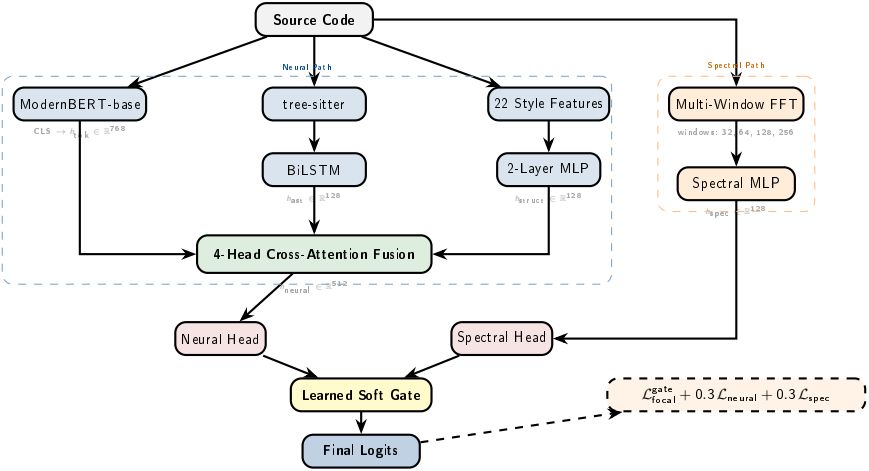
★ DROID: Adversarial

Machine-refined (continue, gap-fill, rewrite). DPO evasion attacks. Varied decoding strategies (temp, top-k, beam).

★ AICD: OOD Stress-Test

77 models incl. reasoning LLMs. Train 3 lang → test 9 lang. Progressive 4-split OOD evaluation.

SpectralCode — Architecture Pipeline



Stream 1: Token Encoder — ModernBERT-base

What is ModernBERT?

- Pre-trained transformer encoder (149M params, **base** variant)
- Trained on massive code + text corpora
- Uses **SDPA** (Scaled Dot-Product Attention) = Flash Attention for speed
- Input: tokenized source code (max 512 tokens)
- Output: `[CLS]` token $\rightarrow h_{\text{tok}} \in \mathbb{R}^{768}$

How It Captures Authorship

The `[CLS]` token aggregates:

- **Lexical patterns**: variable naming, keyword choices
- **Semantic patterns**: how logic is expressed
- **Code idioms**: common patterns per LLM (e.g., GPT tends to use list comprehensions, Llama prefers explicit loops)

Why ModernBERT?

- Outperforms CodeBERT, RoBERTa on code tasks
- SDPA attention = 2x faster than standard attention
- **Base** model (149M) — paper DROID uses **Large** (395M), we match with smaller model!

Limitation

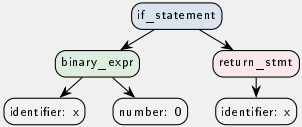
Token-level only — misses structural patterns (nesting depth, control flow) that are visible in the AST.

Stream 2: AST Encoder — tree-sitter + BiLSTM

What is an AST (Abstract Syntax Tree)?

A **tree representation** of code structure. Each node = a syntactic element.

Example: `if x > 0: return x`



What is tree-sitter?

A fast, incremental **parser** that builds ASTs for 40+ programming languages. We support: Python, Java, C++, C, Go, PHP, C#, JS, Rust.

Process:

- Code $\xrightarrow{\text{tree-sitter}}$ AST tree
- DFS traversal → node type sequence
  [if_stmt, binary_expr, identifier, number, return_stmt, ...]
- 67 node types in vocabulary
- BiLSTM encodes sequence → $h_{\text{ast}} \in \mathbb{R}^{128}$

Why AST for authorship?

LLMs have distinct **syntactic preferences**: GPT uses more nested ternary expressions, Llama prefers explicit if-else chains. AST captures these structural "fingerprints."

Stream 3: Structural Features + Stream 4: Spectral FFT Path

22 Structural Features

Handcrafted coding style metrics:

- **Layout**: avg/max line length, indent mean/variance/max
- **Complexity**: #functions, #loops, #conditionals, #try-catch
- **Naming**: snake_case vs camelCase ratio, avg identifier length, single-char var ratio
- **Style**: comment density, empty line ratio, whitespace ratio

$$\xrightarrow{\text{2-layer MLP}} h_{\text{struct}} \in \mathbb{R}^{128}$$

Spectral FFT Path (Key Novelty)

Core insight: Token ID sequences = 1D discrete signals. LLM decoding (top-k, temperature) creates **periodic frequency patterns** invisible in token space.

FFT (Fast Fourier Transform) decomposes signal into frequency components:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-2\pi i \cdot kn/N}$$

Multi-window: 4 scales (32, 64, 128, 256 tokens)
Per window (16 features):

- 8 frequency band energies (low $\rightarrow$ high freq)
- Spectral centroid ("center of mass" of spectrum)
- Rolloff (freq below which 85% energy)
- Flatness (noise-like vs tonal)
- Peak frequency

Fusion: Cross-Attention + Learned Gating

Cross-Attention Fusion (Neural Path)

Three representations enter:

- $h_{\text{tok}} \in \mathbb{R}^{768}$ (token semantics)
- $h_{\text{ast}} \in \mathbb{R}^{128}$ (syntax structure)
- $h_{\text{struct}} \in \mathbb{R}^{128}$ (coding style)

Step 1: Project all to $\mathbb{R}^{512}$
Step 2: Stack as 3-token sequence
Step 3: 4-head self-attention — each view "attends" to others
Step 4: Gated projection $\rightarrow h_{\text{neural}} \in \mathbb{R}^{512}$

Why cross-attention? Allows token features to be *modulated* by AST structure — e.g., "this token pattern is more suspicious given this AST structure."

Learned Soft Gate (Final)

Not all code benefits equally from spectral vs neural analysis.

Gate: $\text{MLP}([h_{\text{neural}}; h_{\text{spec}}]) \rightarrow (w_n, w_s)$

$\text{logits} = w_n \cdot f_n(h_{\text{neural}}) + w_s \cdot f_s(h_{\text{spec}})$

Each path has its **own classification head**. Gate learns per-sample weights.

Training Loss

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{focal}}^{\text{gate}}}_{\text{main}} + \underbrace{0.3\,\mathcal{L}_{\text{focal}}^{\text{neural}}}_{\text{aux}} + \underbrace{0.3\,\mathcal{L}_{\text{focal}}^{\text{spec}}}_{\text{aux}}$$

Focal loss ($\gamma = 2$): down-weights easy samples, focuses on hard ones. Auxiliary losses force each path to be independently useful.

CoDET-M4: SpectralCode vs Paper Baselines (ACL Findings 2025)

Binary Detection (Macro-F1 %)

	P	R	F1	Acc
CatBoost	88.7	88.8	88.8	88.8
CodeBERT	95.7	95.7	95.7	95.7
CodeT5	98.4	98.4	98.4	98.4
UniXcoder	98.7	98.7	98.7	98.7
Ours	**99.1**	**99.1**	**99.1**	**99.1**

Author Attribution (6-class F1)

	F1	Δ
SVM	27.63	—
CatBoost	45.42	—
CodeT5	62.45	—
CodeBERT	64.80	—
UniXcoder	66.33	—
SpectralCode	**69.82**	+3.49

Per-Language Author F1 (%)

	C++	Java	Python
SpectralCode	72.31	70.12	69.22

Per-Source Author F1 (%)

	CF	GitHub	LC
SpectralCode	77.17	56.18	60.35

GitHub is the main bottleneck (↓21% vs CF)

DroidCollection: SpectralCode **Beats** DroidDetect-Base (EMNLP 2025)

Comparison with DROID Paper (Weighted-F1 %)

Type	Method	3-cls	4-cls
Zero-shot	Fast-DetectGPT	64.54	—
	GPTZero	49.10	—
Fine-tuned	CatBoost	72.31	—
	GCN	51.63	—
Full train	GPT-SnifferFT	82.75	—
	CoDet-M4FT	83.25	—
	DroidDetect-B	86.76	92.95
	DroidDetect-L	**88.78**	**94.30**
Ours	**SpectralCode**	**88.77**	**88.02**

All numbers: weighted-F1. DROID paper: domain-avg. Ours: overall.

Key Findings

- ✓ **Beats DroidDetect-Base** (88.77 > 86.76)
- ✓ **Ties DroidDetect-Large** (88.77 ≈ 88.78)
- ✓ Beats **all** zero-shot & fine-tuned
- → Using **base** model (149M) vs paper **large** (395M)!

Scaling Potential

Currently trained on only **100K / 1.06M** (∼1/10) of DROID data.

Full data training expected to improve further.

AICDBench: Ongoing Exploration (EACL 2026)

AICD Paper Baselines vs SpectralCode (Macro-F1 %)

Method	T1 (2-cls)	T2 (12-cls)	T3 (4-cls)
CodeBERT	28.64	23.71	55.60
UniXcoder	25.51	26.64	54.21
DeBERTa	34.13	12.08	54.65
ModernBERT	**30.61**	**32.84**	**61.65**
SpectralCode	29.83	18.93	56.31

Currently trained on 100K / 2.05M (∼1/20) of data

Status: Experimental

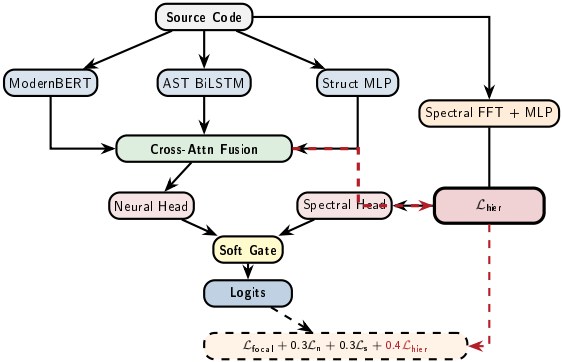
- T1: Competitive with ModernBERT (29.83 vs 30.61)
- T3: Competitive (56.31 vs 61.65)
- T2: Below — need family-aware loss

→ **Using only 1/20 of training data!**

AICD T1 OOD Collapse

Val 99.5% → Test 29.8% **across ALL methods**. 4-split progressive OOD (unseen langs + domains) is fundamentally hard.

HierTreeCode — SpectralCode + Hierarchical Affinity Loss



Hierarchical Affinity Loss

Family Tree (adapts per benchmark):

Qwen	$\rightarrow$ {Qwen1.5, Nxcode}
Meta	$\rightarrow$ {Llama3.1, CodeLlama}
Google	$\rightarrow$ {Gemma, Gemini}

Batch-hard triplet:

$$[\, d_{\textbf{pos}}^{\max} - d_{\textbf{neg}}^{\min} + \alpha \,]_+$$

d: cosine distance on ℓ_2-norm $h_{\textbf{neural}}$
$\alpha = 0.3$ margin

Effect: Forces same-family closer, different-family further apart.

Difference from SpectralCode

SpectralCode = this diagram **without** the red $\mathcal{L}_{\textbf{hier}}$ branch. Everything else is identical.

HierTreeCode: Preliminary Results + Expected Impact

CoDET-M4 Author F1 (✓ Done)

	Author F1	Δ vs UniX
UniXcoder	66.33	—
SpectralCode	69.82	+3.49
HierTreeCode	**70.55**	**+4.22**

Per-generator F1 (HierTreeCode):

Human	GPT	Llama	CodeLlama
0.87	0.92	0.67	0.74

Qwen1.5	Nxcode	Macro
0.44 (+.03)	**0.59**	**70.55**

Why HierTreeCode Will Help Further

- **DROID T3/T4**: Family hierarchy (human → refined → machine → adversarial)
- **AICD T2** (12-class): Groups Google (Gemma+Gemini), Chinese AI (DeepSeek+Qwen)
- **AICD T3**: Adversarial ≈ machine origin
- Family tree adapts per benchmark automatically

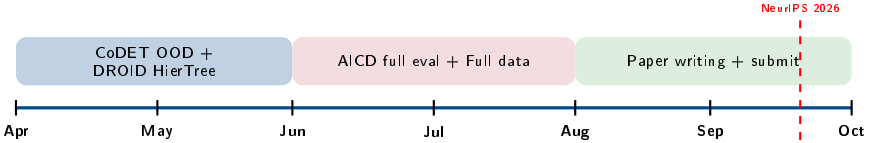
Hierarchical Affinity Loss

$$\mathcal{L}_{\text{hier}} = \frac{1}{|\mathcal{A}|} \sum_i [d_{i,p^*} - d_{i,n^*} + \alpha]_+$$

p^*: farthest same-family (hard positive)
n^*: closest diff-family (hard negative)
Cosine distance on ℓ_2-normalized h_{neural}

Full Evaluation Matrix

	CoDET-M4 ACL'25	DROID EMNLP'25	AICD EACL'26
Binary	✓ 99.06%	✓ 88.77%^{w} T3	🕒 29.83% T1
Attribution	✓ 70.55% (6-cls)	—	🕒 T2 (12-cls)
Refined	—	✓ 88.77%^{w} T3	🕒 T3
Adversarial	—	✓ 88.02%^{w} T4	🕒 T3
OOD Lang	🕒 LOO	—	🕒 3→9 lang



Contributions & Summary

Technical Contributions

1. **SpectralCode**: Multi-stream (token + AST + structural + spectral FFT) with cross-attention + gating — *SOTA on CoDET-M4, beats DroidDetect-Base on DROID*
2. **HierTreeCode**: First use of LLM genealogy as structured prior — encodes parent-child as embedding distance constraints via batch-hard triplet mining
3. **Unified 3-benchmark evaluation**: First work covering CoDET-M4 + DROID + AICD — binary + attribution + OOD + adversarial in one method
4. **Scaling insight**: Competitive results using only 1/10–1/20 of training data — full data expected to close remaining gaps

Results So Far

	CoDET-M4	DROID	AICD
Binary	99.06% (+0.41 vs UniX)	88.77%^{w} (>DroidDet-B)	⏲ ongoing
Author	70.55% (+4.22 vs UniX)	—	⏲ T2
Data used	100K/500K	100K/1.06M	100K/2.05M

CoDET-M4: 10 Methods Explored — What Worked and What Didn't

Full Leaderboard (Author 6-class F1)

#	Method	Author F1	Key Idea
1	**HierTreeCode**	**70.55**	Family tree loss
2	RAGDetect	70.46	Retrieval-augmented
3	BiScopeCode	70.20	MLM memorization
4	TTACode	70.20	Test-time adapt
5	MultiGranCode	70.19	CharCNN fusion
6	GroupDRO	70.17	Worst-group opt
7	SelfDistillCode	70.14	EMA distillation
8	ProtoCon	70.13	Prototype + unif
9	SpectralCode	69.82	FFT spectral
10	GraphStyleCode	69.71	GAT on AST
—	EAGLECode	62.89	DANN (failed)

DANN Catastrophically Fails

Gradient reversal forces generator-invariant features — this *directly opposes* attribution.
Qwen1.5 F1 drops to **0.198** (random).
Lesson: Domain adversarial $\neq$ attribution.

Key Insights

- All methods cluster in 69.7–70.6% author F1
- Qwen1.5/Nxcode confusion is THE bottleneck
- GitHub source ↓21% vs CodeForces
- GAT on AST provides no benefit
- Spectral FFT is surprisingly effective

AICD + DROID: 13 Deep Methods — Cross-Benchmark Insights

Leaderboard (5 completed methods)

#	Method	AICD	DROID	Avg
1	SpectralCode	.350	.847	**.549**
2	TokenStat	.330	**.852**	.539
3	MoE-Code	.324	.849	.534
4	SlotCode	.329	.839	.533
5	DomainMix	.286	.716	.458

Methods That Failed

CausAST: Ortho → 0 (worst) | **OSCP**: Whitening dominates | **AST-IRM**: Penalty explodes to 5000

Cross-Benchmark Insights

1. **AICD T1 OOD unsolved**: Val 99.5% → Test 30% for ALL methods
2. **Spectral features transfer best**: Wins overall despite simplicity
3. **Token stats are powerful**: Entropy, burstiness, Yule-K beat complex architectures on DROID
4. **Mixup helps OOD**: DomainMix achieves best AICD T1 via embedding mixup
5. **Only 1/10–1/20 data used**: Full data → higher scores expected

Thank You

Questions & Discussion

CoDET-M4 (ACL Findings'25) · DroidCollection (EMNLP'25) · AICDBench (EACL'26)

Code and experiments available upon request